

MSDM5004 Spring 2026

Project 1 Solution

0. Definitions

In both questions, J is the numbers of steps in space x , and N is the number of steps in time t . Therefore, the grid of the approximation U uses $J + 1$ points in x and $N + 1$ points in t . Then for programming simplicity, let the indices of the points follow the convention of MATLAB and start from 1. It means that

$$U_j^n \approx u((j-1)\Delta x, (n-1)\Delta t)$$

for $1 \leq j \leq J + 1$ and $1 \leq n \leq N + 1$.

However, you are not allowed to construct a $(N+1) \times (J+1)$ array U to save the results of all time steps. This is an essential technique in real-world calculations because N is typically a big number.

1. Numerical methods for partial differential equations (parabolic PDEs)

(1) The explicit scheme

$$U_j^{n+1} = U_j^n + a\mu(U_{j+1}^n - 2U_j^n + U_{j-1}^n)$$

can be implemented in MATLAB as

```
J = 20; dx = 0.1; x = -1:dx:1;
N = 50; dt = 0.0025; % or dt = 0.0026
a = 2; mu = dt/dx^2;
u0 = @(x) (x<=1/2).*(1/3*(x+1)) + (x>1/2).*(1-x);
Up = u0(x); % Up(j) = U(n+1, j), a 1*(J+1) array
Us = []; % Us will save U(1, :), U(2, :), U(26, :), and U(51, :),
for n=1:N % i.e. the results that we actually want to plot
    Un = Up; % Un(j) = U(n, j), also a 1*(J+1) array
    if n==1 | n==2 | n==26; Us = [Us; Un]; end
    for j=2:J
        Up(j) = Un(j) + a*mu*( Un(j+1)-2*Un(j)+Un(j-1) );
    end
end; Us = [Us; Up]; % here Up = U(51, :) as it is evaluated at n=N=50
```

Notice that this implementation does not save everything but keeps track of the approximations of two steps only, i.e. $U_n(j)$ and $U_p(j)$.

In addition, the above code works for (i) only, and you should change dt=0.0025 to dt=0.0026 for (ii). Their solutions are plotted after discussing (2).

(2) The Crank-Nicolson method solves

$$U_j^{n+1} - U_j^n = \frac{a\mu}{2} \left((U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}) + (U_{j+1}^n - 2U_j^n + U_{j-1}^n) \right),$$

which is equivalent to

$$-\frac{a\mu}{2}U_{j-1}^{n+1} + (1 + a\mu)U_j^{n+1} - \frac{a\mu}{2}U_{j+1}^{n+1} = \frac{a\mu}{2}U_{j-1}^n + (1 - a\mu)U_j^n + \frac{a\mu}{2}U_{j+1}^n$$

and thus

$$\begin{cases} (1 + a\mu)U_2^{n+1} - \frac{a\mu}{2}U_3^{n+1} &= (1 - a\mu)U_2^n + \frac{a\mu}{2}U_3^n \\ -\frac{a\mu}{2}U_2^{n+1} + (1 + a\mu)U_3^{n+1} - \frac{a\mu}{2}U_4^{n+1} &= \frac{a\mu}{2}U_2^n + (1 - a\mu)U_3^n + \frac{a\mu}{2}U_4^n \\ \vdots &\vdots \\ -\frac{a\mu}{2}U_{J-2}^{n+1} + (1 + a\mu)U_{J-1}^{n+1} - \frac{a\mu}{2}U_J^{n+1} &= \frac{a\mu}{2}U_{J-2}^n + (1 - a\mu)U_{J-1}^n + \frac{a\mu}{2}U_J^n \\ -\frac{a\mu}{2}U_{J-1}^{n+1} + (1 + a\mu)U_J^{n+1} &= \frac{a\mu}{2}U_{J-1}^n + (1 - a\mu)U_J^n \end{cases}.$$

Remember $U_1^n = U_{J+1}^n = 0$ because $u(-1, t) = u(1, t) = 0$. The system can then be rewritten as

$$\begin{pmatrix} d_2 & s & & & 0 \\ s & d_3 & s & & \\ & \ddots & \ddots & \ddots & \\ & & s & d_{J-1} & s \\ 0 & & & s & d_J \end{pmatrix} \begin{pmatrix} U_2^{n+1} \\ U_3^{n+1} \\ \vdots \\ U_{J-1}^{n+1} \\ U_J^{n+1} \end{pmatrix} = \begin{pmatrix} b_2 \\ b_3 \\ \vdots \\ b_{J-1} \\ b_J \end{pmatrix},$$

where

$$d_j = 1 + a\mu, \quad s = -\frac{a\mu}{2}, \quad \text{and} \quad b_j = \frac{a\mu}{2}U_{j-1}^n + (1 - a\mu)U_j^n + \frac{a\mu}{2}U_{j+1}^n$$

for $2 \leq j \leq J$. The Thomas algorithm solves it by first updating

$$d_j \leftarrow d_j - \frac{s^2}{d_{j-1}} \quad \text{and} \quad b_j \leftarrow b_j - \frac{s}{d_{j-1}}b_{j-1}$$

for j from 3 to J sequentially and then evaluating

$$U_J^{n+1} = \frac{b_J}{d_J} \quad \text{and} \quad U_j^{n+1} = \frac{b_j - sU_{j+1}^{n+1}}{d_j}$$

for j from $J - 1$ to 2 sequentially. All the steps above can be implemented in MATLAB as

```

J = 20; dx = 0.1; x = -1:dx:1;
N = 50; dt = 0.0025; % or dt = 0.0026
a = 2; mu = dt/dx^2;
u0 = @(x) (x<=1/2).*(1/3*(x+1)) + (x>1/2).*(1-x);
Up = u0(x);
Us = [];
for n=1:N
    Un = Up;
    if n==1 | n==2 | n==26; Us = [Us; Un]; end
    d = ones(1, J) * (1+a*mu); % d(j): d(1) is a dummy
    s = -a*mu/2;
    b = @(j) a*mu/2*Un(j-1) + (1-a*mu)*Un(j) + a*mu/2*Un(j+1);
    b = [1 b(2:J)]; % b(j): b(1) is a dummy
    for j=3:J
        d(j) = d(j) - s^2/d(j-1);
        b(j) = b(j) - s/d(j-1)*b(j-1);
    end
    Up(J) = b(J)/d(J);
    for j=J-1:-1:2
        Up(j) = ( b(j)-s*Up(j+1) )/d(j);
    end
end; Us = [Us; Up];

```

Again, the above suits (i) only, and you should change dt=0.0025 to dt=0.0026 for (ii).

The four previous solutions U are plotted as Fig. 1(1i, 1ii, 2i, 2ii) with the following code.

```

f = figure; f.Position(3:4) = [600 300];
plot(x, Us(1, :), 'o-'); hold on
plot(x, Us(2, :), 's--');
plot(x, Us(3, :), '^-.');
plot(x, Us(4, :), 'v:'); hold off
ls = {'$t=0$', '$t=\Delta t$', '$t=25\Delta t$', '$t=50\Delta t$'};
legend(ls, 'Interpreter', 'latex', 'Location', 'northwest');
xlabel('$x$', 'Interpreter', 'latex');
ylabel('$U$', 'Interpreter', 'latex');
ylim([0 0.6]);

```

The computed points of U are represented by the markers, whereas the connecting lines are merely for visual aid. Observe (1) how the explicit scheme becomes unstable once $a\mu > \frac{1}{2}$ and (2) how the Crank-Nicolson method is unconditionally stable.

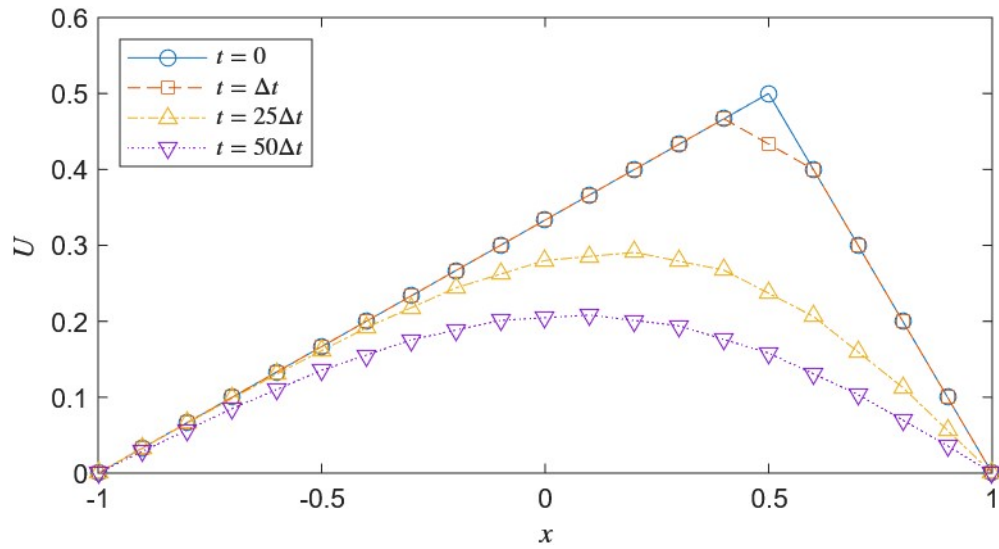


Fig. 1(i). The explicit scheme with $\Delta t = 0.0025$.

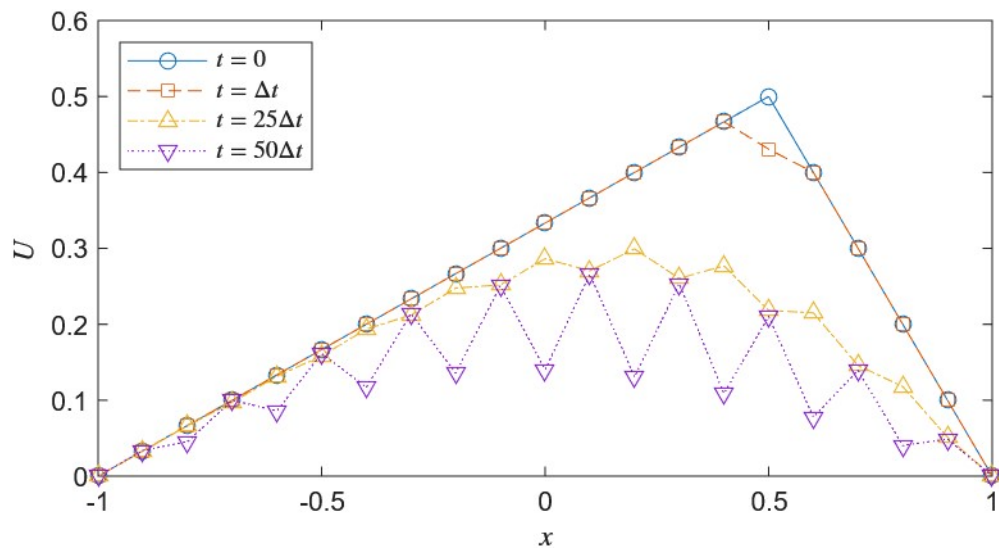


Fig. 1(ii). The explicit scheme with $\Delta t = 0.0026$.

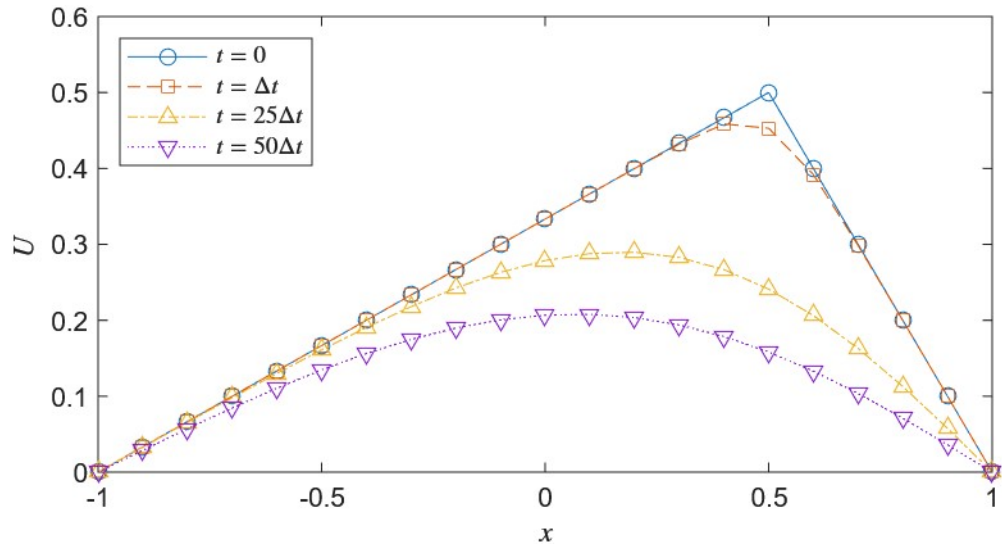


Fig. 1(2i). The Crank-Nicolson method with $\Delta t = 0.0025$.

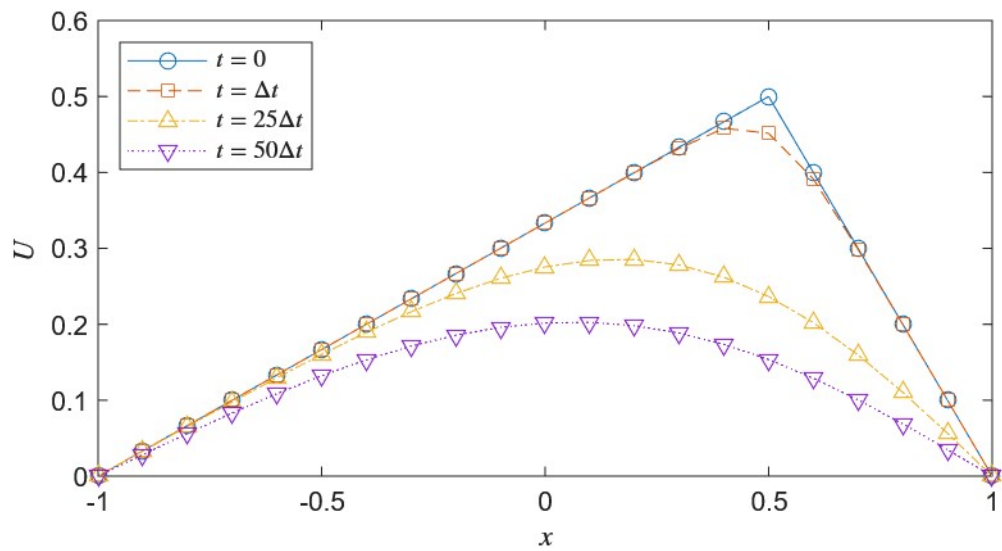


Fig. 1(2ii). The Crank-Nicolson method with $\Delta t = 0.0026$.

2. Numerical methods for partial differential equations (hyperbolic PDEs)

- (1) Since $a = 1.8 > 0$, the upwind method solves

$$U_j^{n+1} = U_j^n - a\nu(U_j^n - U_{j-1}^n) .$$

It can be implemented in MATLAB as

```
a = 1.8; nu = 0.25; dt = 0.01; % or dt = 0.0025
dx = dt/nu; x = -2:dx:2; N = 0.5/dt; J = 4/dx;
u0 = @(x) (x<=0)*1; % *1 for converting the class of u0
Up = u0(x); % from logical to double
for n=1:N
    Un = Up;
    for j=2:J
        Up(j) = Un(j) - a*nu*( Un(j)-Un(j-1) );
    end
end
```

The above code works for (i) $\Delta t = 0.01$ only, and you should change dt=0.01 to dt=0.0025 for (ii) $\Delta t = 0.0025$. Their solutions are plotted in (3).

- (2) The Lax-Wendroff method solves

$$U_j^{n+1} = U_j^n - \frac{a\nu}{2}(U_{j+1}^n - U_{j-1}^n) + \frac{(a\nu)^2}{2}(U_{j+1}^n - 2U_j^n + U_{j-1}^n) .$$

It can be implemented in MATLAB in

```
a = 1.8; nu = 0.25; dt = 0.01; % or dt = 0.0025
dx = dt/nu; x = -2:dx:2; N = 0.5/dt; J = 4/dx;
u0 = @(x) (x<=0)*1;
Up = u0(x);
for n=1:N
    Un = Up;
    for j=2:J
        Up(j) = Un(j) - a*nu/2*( Un(j+1)-Un(j-1) ) ...
            + (a*nu)^2/2*( Un(j+1)-2*Un(j)+Un(j-1) );
    end
end
```

Again, the above suits (i) $\Delta t = 0.01$ only, and you should change dt=0.01 to dt=0.0025 for (ii) $\Delta t = 0.0025$. Their solutions are also plotted in (3).

(3) The exact solution is

$$u(x, t) = u_0(x - at) = \begin{cases} 1 & (x - at \leq 0) \\ 0 & (x - at > 0) \end{cases}.$$

The four previous numerical solutions U are plotted atop the exact solution as Fig. 2(1i, 1ii, 2i, 2ii) with the following code. The computed points of U are represented by the circle markers, whereas the connecting lines are merely for visual aid.

```
f = figure; f.Position(3:4) = [600 300];
plot(x, u0(x-a*0.5), '-'); hold on
plot(x, Up, 'o-', 'Markersize', 4); hold off
title('$t=0.5$', 'Interpreter', 'latex');
legend({'exact solution $u$', 'numerical solution $U$'}, ...
       'Location', 'southwest', 'Interpreter', 'latex');
xlabel('$x$', 'Interpreter', 'latex');
ylim([-0.1 1.4]);
```

One can observe several things from the graphs.

- At $t = 0.5$, $x - at = 0$ gives $x = 0.9$, so a discontinuity appears in the exact solution u at $x = 0.9$. All numerical solutions U can capture this abrupt change.
- For both methods, a smaller Δt makes U change more abruptly at $x = 0.9$ and thus approximate u better.
- The upwind method smooths out the corners of the discontinuity significantly, turning it into a gradual change. In contrast, the Lax-Wendroff method captures the abrupt change better than the upwind method.
- However, the Lax-Wendroff method generates oscillations near the discontinuity.

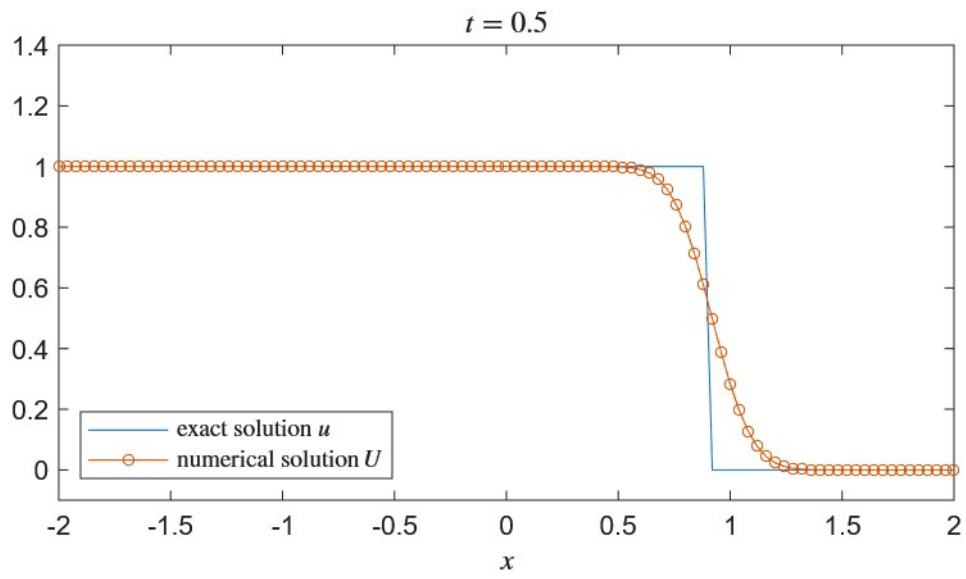


Fig. 2(1i). The upwind method with $\Delta t = 0.01$.

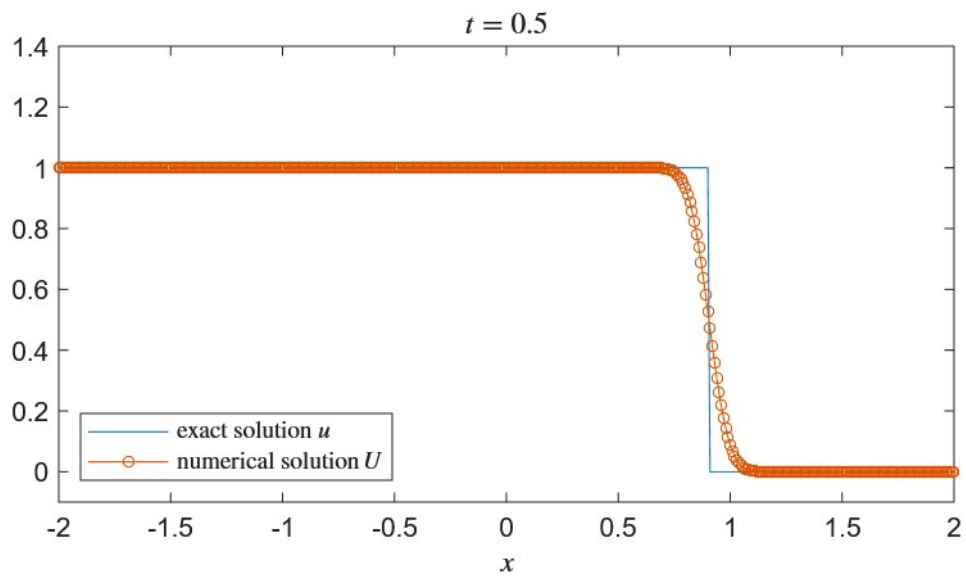


Fig. 2(1ii). The upwind method with $\Delta t = 0.0025$.

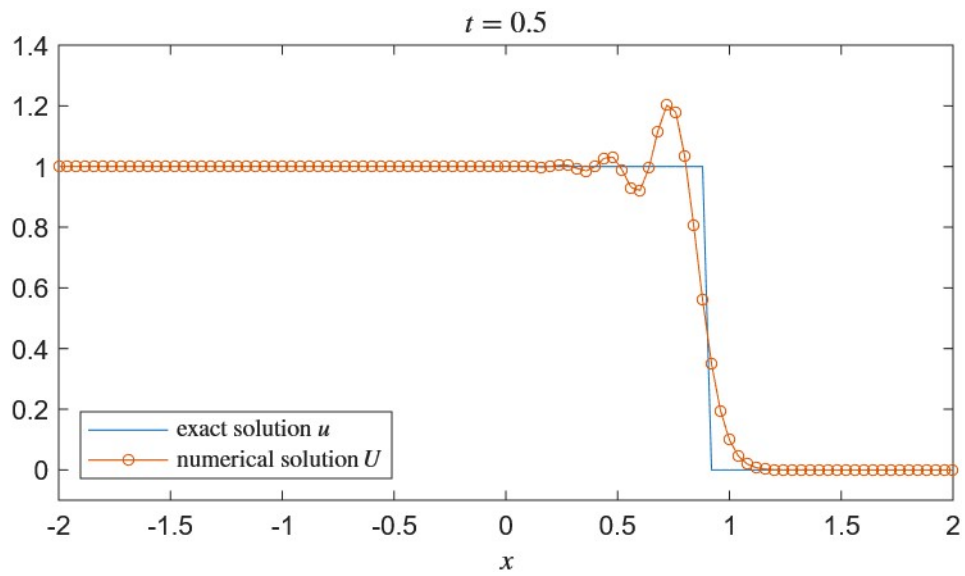


Fig. 2(2i). The Lax-Wendroff method with $\Delta t = 0.01$.

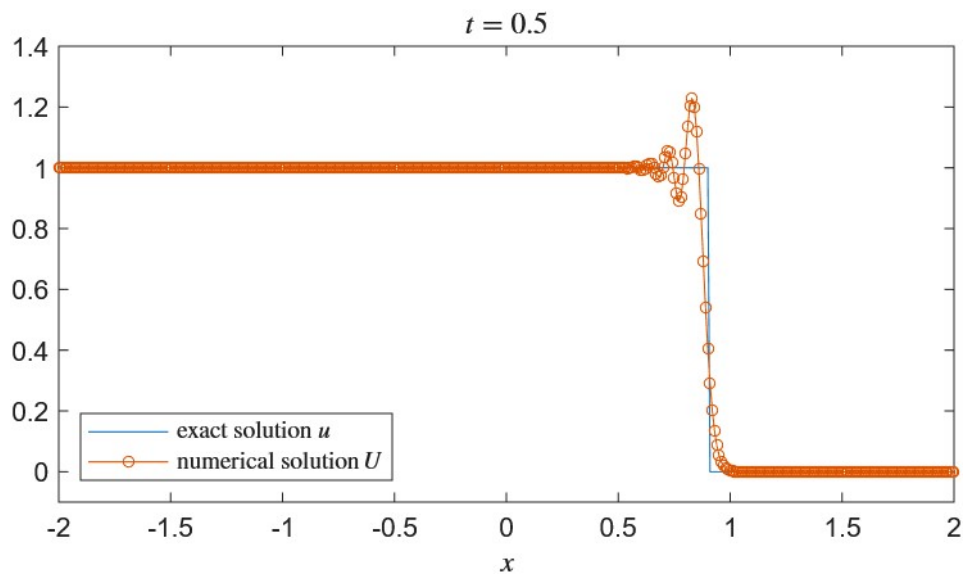


Fig. 2(2ii). The Lax-Wendroff method with $\Delta t = 0.0025$.